https://habr.com/ru/articles/985400/

Статья на Хабре под номером 985400 вышла совсем недавно (17 января 2026) и называется:

«Альтернативы диаграмме Ганта для дорожных карт продукта»

Автор — PProduct.

Это практический материал именно про продуктовый менеджмент: как лучше визуализировать и коммуницировать **product roadmap**, когда классическая диаграмма Ганта уже не подходит (слишком жесткая, негибкая для современных продуктовых команд, плохо отражает неопределенность и приоритеты).

Основные идеи и альтернативы, которые разбираются

1. **Now–Next–Later**
   Самый популярный и часто рекомендуемый вариант в статье.
   - Now — то, что делаем прямо сейчас (1–3 месяца).
   - Next — следующий горизонт (3–6/9 месяцев).
   - Later — всё остальное, что важно, но не горит.
     Плюсы: очень простая, понятная стейкхолдерам, не требует точных дат, хорошо работает с изменяющимися приоритетами.
2. **Goal Oriented / Outcome-based roadmaps**
   Фокус не на фичах, а на целях / результатах (часто через OKR).
   Даты и конкретные задачи вторичны — главное «что хотим достичь и зачем».
3. **Theme-based / Topic-based roadmaps**
   Группировка по крупным темам/эпикам/направлениям (например, «Улучшение онбординга», «Монетизация 2.0», «Performance & Scalability»).
4. **Prioritization matrices + визуализация типа «Ковёр приоритетов» или «Impact/Effort» + timeline-lite**.

Статья подчёркивает, что в продуктовой разработке 2025–2026 годов Гант чаще вредит, чем помогает, потому что:

- создаёт ложное ощущение точности сроков
- демотивирует команду при сдвигах

- плохо масштабируется на discovery / research / эксперименты

Вместо этого предлагается переходить на более гибкие форматы + обязательно привязывать roadmap к метрикам / OKR / North Star.

Если коротко: автор агитирует почти всех отказаться от Ганта в пользу Now-Next-Later + Goal Oriented подхода — это сейчас один из самых живучих и рекомендуемых паттернов в продуктовых командах (особенно в компаниях, которые уже перешли на shape up / dual-track agile и т.п.).

Тебе интересно что-то конкретное из статьи — примеры картинок/шаблонов, как внедрять Now-Next-Later в Jira/Confluence, или сравнение с другими подходами (типа Opportunity Solution Tree, Wardley Mapping и т.д.)? Могу углубиться. 😄

Расскажи. Вот я сам себе придумал небольшой проект. И хочу его закончить к определенной дате. Это разработка маленькой программы на flutter для изучения языка. Ограничение по времени мне необходимо. Иначе растяну на бесконечность. MVP должно быть готово к этой дате. Но понятно что гант для общего планирования. Чтобы в первую очередь вообще понять из чего что-то т проект, какие задачи / этапы и как они связаны. Для моего маленького проекта как рекомендуется вести его, самые современные техники? У тебя есть доступ к нашим перепискам? Я говорю про проект gitdoit

Размышление на протяжении 33s

Да, Андрей, у меня есть доступ к нашим предыдущим перепискам (через поиск по истории разговоров), и я помню твой проект GitDoIt — это простое Flutter-приложение для управления to-do lists через интеграцию с GitHub (синхронизация с issues, аутентификация, базовый UI). Ты упоминал, что это для изучения Flutter и Dart, и мы обсуждали breakdown задач, структуру кода (BLoC или Riverpod для state management) и даже примерный timeline в 2 недели для MVP. Если что-то изменилось в идее, уточни, но я опираюсь на это.

Поскольку проект маленький, личный и с фокусом на обучение, жёсткий Гант (как в той статье на Habr, которую мы обсуждали) действительно подойдёт только для high-level обзора — чтобы визуализировать этапы и зависимости (например, сначала setup окружения, потом UI, потом API-интеграция). Но для повседневного ведения он может быть слишком ригидным и демотивирующим, если сроки сдвинутся из-за экспериментов с языком. Вместо этого современные подходы (на 2025–2026 годы) для соло-разработчиков подчёркивают гибкость, итерации и интеграцию с инструментами, которые минимизируют overhead. Вот

как рекомендуется вести такой проект, с акцентом на time-boxing (чтобы не растягивать) и актуальные техники:

1. Общий подход: Гибкий Agile для одного (Solo Agile или Indie Hacking style)

- **Почему?** В 2026 году для маленьких проектов (особенно образовательных) — это не тяжёлые фреймворки вроде Scrum с полным ceremony, а упрощённые версии, где ты сам себе product owner, developer и tester. Фокус на быстрых итерациях, чтобы поддерживать мотивацию и учиться на ходу.
- **Как применить к GitDoIt:**
  - Разбей на короткие циклы: например, 1–2 недели на "shape" (планирование) + build + review. Это из Shape Up (метод от Basecamp, очень популярен сейчас для инди-проектов).
  - Установи time-box: фиксированную дату для MVP (скажем, 2–4 недели от старта). Внутри — daily limits, например, 2–4 часа в день, чтобы избежать burnout.
  - Интегрируй обучение: каждый цикл заканчивай мини-ретроспективой (что узнал по Dart/Flutter? Что пошло не так?).

2. Планирование и Roadmap: Альтернативы Ганту

- Как мы говорили по статье на Habr, для продуктов (даже маленьких) лучше Now-Next-Later или Goal-Oriented подходы. Для твоего случая:
  - **Now-Next-Later Roadmap:**
    - **Now:** Базовый setup и core фичи (установка Flutter, простой UI для to-do, GitHub auth). Дедлайн: 1 неделя.
    - **Next:** Интеграция с GitHub API (синхронизация issues), базовое тестирование. Дедлайн: +1–2 недели.
    - **Later:** Polish (UI/UX улучшения, edge cases, deploy на Android/iOS). Без жёстких дат, чтобы не давить.
  - **Goal-Oriented:** Привяжи к целям, а не фичами. Например, "MVP: Авторизоваться в GitHub и создать/просмотреть to-do как issue. Метрика успеха: Работает на моём устройстве к [дата]".
  - **Инструменты для визуализации:**
    - Notion или Miro для mind maps/roadmaps (бесплатно, легко рисовать связи задач).

    - GitHub Projects: Встроенный в repo, можно сделать Kanban-борду с колонками и связями (dependencies via issues).
    - Если хочешь автоматизацию — Linear или ClickUp (бесплатные для соло, с AI-подсказками задач в 2026).

Для понимания задач/этапов:

- Начни с breakdown: Список этапов для GitDoIt (на основе наших прошлых обсуждений):
  1. **Setup:** Установить Flutter SDK, создать проект, настроить Git repo. (Зависит: ничего, 1–2 дня).
  2. **UI Basics:** Материал дизайн, экраны для login и to-do list. (Зависит: от setup, используй widgets как ListView).
  3. **API Integration:** OAuth с GitHub, GraphQL/REST для issues (пакеты: oauth2, graphql_flutter). (Зависит: от UI).
  4. **Sync Logic:** Логика pull/push to-do. (Зависит: от API).
  5. **Testing & MVP:** Unit tests (flutter_test), ручное тестирование, build APK/IPA.
- Визуализируй в Ганте только для старта (в Google Sheets или Draw.io), потом перейди на гибкий формат.

3. Трекинг задач: Kanban + Time-Boxing

- **Kanban для соло:** Самая рекомендуемая техника сейчас — простая доска с колонками "To Do", "In Progress", "Done". Лимит WIP (work in progress) — не больше 2–3 задач одновременно, чтобы не распыляться.
  - Инструменты: Trello (визуально, с deadlines), GitHub Issues (интегрировано с кодом, labels для приоритетов как High/Low).
  - Добавь time estimates: Каждая задача — с оценкой в часах (Pomodoro-style: 25 мин работы + перерыв).
- **Time-Boxing:** Чтобы избежать бесконечности, фиксируй слоты. Например, "Сегодня — только UI, 3 часа max". Используй таймеры вроде Focus Booster или встроенные в VS Code (Flutter dev в VS Code или Android Studio).
- **AI-ассист:** В 2026 популярно использовать AI для генерации задач/кода. Например, GitHub Copilot для Flutter snippets, или меня — для breakdown или code reviews.

4. Код и Dev Practices: Современные для Flutter

- **State Management:** Riverpod 2.x (более современный и declarative, чем BLoC; популярен в 2026 для маленьких apps).
- **Структура:** Feature-first: lib/features/auth, lib/features/todos — с services, models, UI.
- **CI/CD:** Даже для маленького — настрой GitHub Actions для auto-build/test на push.
- **Learning Focus:** Каждый commit — с описанием "Что узнал" (git commit -m "Added auth: learned about async/await in Dart").
- **Версионирование:** Semantic Versioning для MVP (0.1.0).

5. Мотивация и Риски

- Чтобы не растянуть: Ежедневные check-in (утром: "Что сегодня?"). Weekly review: Что сделано vs план.
- Если сдвиг — адаптируй, не демотивируйся (гибкость — ключ в современных методах).
- Ресурсы: Официальная Flutter doc (flutter.dev), YouTube-каналы вроде Reso Coder или Flutter Community (актуальные на 2026 с AI-интеграциями).

Если хочешь, давай углубим: нарисуем конкретный план для GitDoIt (с датами, если укажешь deadline), сгенерируем шаблон задач в Notion-style, или даже code snippets для старта. Что скажешь? 😊

> У меня есть доступ к курсу Udemy - Flutter & Dart - The Complete Guide [2025 Edition] я хочу вплести его в мой проект. Как учебные материалы. Я ещё купить одну книгу, но пока нет денег, пришлось отказаться от доставки.
> Можешь соотнести этот курс по плану обучения с моим приложением? Чтобы сопоставить главу (блок) курса и блок моего плана .
> Хочу, давай углубим, нарисуем план (возьми даты из нашей беседы) шаблон задач можно сгенерировать для logseq я пользуюсь им. Можно и код сниппетс. Так как я уже забыл то что выучил во флаттер. И надо ещё раз начать изучать. Хочу погрузиться и разобраться как работает

Размышление на протяжении 41s

Андрей, круто, что у тебя есть этот курс на Udemy — "Flutter & Dart - The Complete Guide [2025 Edition]" (на самом деле, по актуальным данным, это обновлённая

версия до 2026, но содержание очень близкое). Это один из топовых курсов от Maximilian Schwarzmüller, с фокусом на практике через мини-проекты (типа dice app, quiz, expense tracker, meals app и todo app). Он идеально вписывается в твой GitDoIt, потому что курс строится от основ Dart/Flutter к реальным приложениям, включая state management, навигацию и базовую работу с данными (futures для async, что подойдёт для GitHub API).

Ты хочешь освежить знания и погрузиться заново — отлично, курс как раз для этого. Я соотнёс его разделы (sections) с блоками твоего плана для GitDoIt. Курс имеет 16+ секций, но я фокусируюсь на релевантных (Dart basics, widgets, state, navigation, API/async). Если в курсе чего-то не хватает (например, глубокого OAuth для GitHub), можно дополнить официальной докой Flutter или моими snippets.

Соотнесение курса с планом GitDoIt

Твой план (на основе наших бесед):

1. **Setup** (установка, проект creation).
2. **UI Basics** (экраны, widgets для to-do list и login).
3. **API Integration** (OAuth auth с GitHub, GraphQL/REST для issues).
4. **Sync Logic** (pull/push to-do как issues).
5. **Testing & MVP** (тесты, build, deploy).

Соотнесение с курсом (sections по syllabus):

- **Setup → Section 1: Introduction (13 лекций, ~1ч)**: Полностью покрывает. Установка Flutter SDK, создание первого проекта, running app. Начни здесь, чтобы освежить setup. (Параллельно: создай repo для GitDoIt на GitHub.)
- **UI Basics → Sections 2-4: Basics I/II + Interactivity/Widgets/Theming (39+39+37 лекций, ~11ч)**: Идеально. Section 2 (Roll Dice App) — базовые widgets, functions, stateful widgets. Section 3 (Quiz App) — conditional rendering, lists, mapping data. Section 4 (Expense Tracker) — TextField input, ListView для списков (как твой to-do), theming (light/dark mode). Примени: Построй базовый UI для to-do list, добавь input для новых задач.
- **API Integration → Section 4 (частично) + Более поздние (Networking, если есть в полной версии)**: В Section 4 есть "Working with Futures" для async data (основа для API calls). Курс имеет разделы на HTTP requests (fetch data from internet), auth features (OAuth basics). Если не хватает деталей для GitHub —

используй пакеты как oauth2 и graphql_flutter (я дам snippets). Параллельно: Интегрируй auth и базовый API call.

- **Sync Logic → Sections 5-6: Multi-Screen Apps + App-wide State (28+16 лекций, ~4.5ч)**: Section 5 (Meals App) — navigation, passing data между экранами (для login → to-do screen). Section 6 — Riverpod для state management (рекомендую для GitDoIt, как мы обсуждали). Примени: Управляй состоянием to-do (local + sync с GitHub issues).
- **Testing & MVP → Section 7: Flutter & Dart Internals (9 лекций, ~0.7ч) + Дополнительные (Testing, если в курсе)**: Покрывает widget lifecycle, keys, performance. Курс имеет базовые тесты (flutter_test). Для MVP: Добавь unit tests для sync logic.

Общий совет: Проходи курс параллельно проекту — после каждой section применяй на GitDoIt. Общее время курса ~30ч, но для твоего MVP хватит первых 7 sections (~25ч). Если забыл — стартуй с Section 1-2 для refresh Dart (variables, functions, classes).

Подробный план с датами

Из наших бесед: примерный timeline 2 недели для MVP (MVP: Авторизация, базовый to-do list с sync issues на GitHub, работает на устройстве). Текущая дата — 18 января 2026 (Amadora time). Предполагаю старт сегодня (18 Jan), MVP к 1 февраля 2026 (14 дней, с буфером на выходные). Каждый день — 2-4ч (time-boxing, чтобы не растянуть). Циклы: Ежедневно курс + практика в GitDoIt.

- **Дни 1-2 (18-19 Jan): Setup & Dart Refresh**
  Курс: Section 1 + начало 2.
  GitDoIt: Установка, create project, basic scaffold. Goal: Запустить empty app.
- **Дни 3-5 (20-22 Jan): UI Basics**
  Курс: Sections 2-3 (widgets, state, lists).
  GitDoIt: Построй login screen и to-do list UI (ListView, TextField). Goal: Интерактивный UI без backend.
- **Дни 6-8 (23-25 Jan): API Integration & Auth**
  Курс: Section 4 (futures, input). Дополни докой: pub.dev/packages/oauth2.
  GitDoIt: Добавь GitHub OAuth, базовый API call для user info. Goal: Авторизация работает.
- **Дни 9-11 (26-28 Jan): Sync Logic & State**
  Курс: Sections 5-6 (navigation, Riverpod).

GitDoIt: Интеграция с GitHub issues (create/view/sync to-do). State management для списка. Goal: Полный sync.

- **Дни 12-14 (29-31 Jan): Testing & Polish**
  Курс: Section 7 (internals).
  GitDoIt: Unit tests, fix bugs, build APK. Goal: MVP ready к 1 Feb.
- **After MVP (Later):** Polish UI, edge cases, deploy (если хочешь на stores).

Ежедневно: Mini-review (что узнал? Commit в git). Если сдвиг — адаптируй, но держи deadline.

Шаблон задач для Logseq

Logseq — отличный выбор для outlining. Вот шаблон в формате Logseq (markdown с блоками). Скопируй в новую страницу, например [[GitDoIt Plan]]. Используй TODO/DONE, DEADLINE, и подблоки для subtasks. Я добавил ссылки на курс sections (предполагая, у тебя доступ). Можно добавить properties как :: priority high.

```text
- TODO Setup & Dart Refresh #gitdoit
  DEADLINE: <2026-01-19 Sun>
  - Изучить Section 1 курса (Udemy link: Introduction)
  - Установить Flutter SDK и создать проект GitDoIt
  - Запустить первый app и commit в GitHub repo
  - Subtask: Освежить Dart basics (functions, variables) из Section 2

- TODO UI Basics #gitdoit
  DEADLINE: <2026-01-22 Thu>
  - Пройти Sections 2-3 курса (Basics I/II, Quiz App)
  - Создать login screen с TextField и Button
  - Добавить ListView для to-do items
  - Тестировать UI на эмуляторе (Android/iOS)
  - Subtask: Добавить theming (dark mode) из Section 4

- TODO API Integration & Auth #gitdoit
  DEADLINE: <2026-01-25 Sun>
  - Изучить futures/async в Section 4
  - Добавить пакет oauth2 в pubspec.yaml
  - Реализовать GitHub OAuth login
  - Тестовый API call: Получить user repos или issues
  - Subtask: Обработать errors (dialogs из курса)

- TODO Sync Logic & State #gitdoit
  DEADLINE: <2026-01-28 Wed>
  - Пройти Sections 5-6 (Navigation, Riverpod)
  - Установить Riverpod для state management
```

```
  - Логика: Pull issues как to-do, push new to-do как issue
  - Добавить navigation: Login -> ToDo screen
  - Subtask: Manage app-wide state для to-do list

- TODO Testing & MVP #gitdoit
  DEADLINE: <2026-02-01 Sat>
  - Изучить Section 7 (Internals)
  - Написать unit tests для sync и auth (flutter_test)
  - Fix bugs, build APK/IPA
  - Review: Что узнал по Flutter? Deploy на устройство
  - Subtask: Optional polish (icons, loading indicators)

- LATER Post-MVP Improvements #gitdoit
  - Добавить offline support (hive или sqflite из advanced sections курса)
  - Улучшить UI (animations из later sections)
```

В Logseq это отрендерится как collapsible blocks. Добавь journaling: Каждый день лог в [[Journal/2026-01-18]] с прогрессом.

Code Snippets для refresh и старта

Чтобы погрузиться заново — вот базовые snippets на Dart/Flutter. Начни с main.dart. (Предполагаю VS Code с Flutter extension.)

1. **Basic Setup (main.dart)** — Основа app из Section 1-2.

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'GitDoIt',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('GitDoIt To-Do')),
      body: const Center(child: Text('Hello, Andrey! Start building
UI.')),
    );
  }
}
```

2. **Stateful Widget с List (для to-do, из Section 3-4).**

```Dart
class ToDoScreen extends StatefulWidget {
  const ToDoScreen({super.key});

  @override
  _ToDoScreenState createState() => _ToDoScreenState();
}

class _ToDoScreenState extends State<ToDoScreen> {
  final List<String> _todos = ['Task 1', 'Task 2']; // Dummy data
  final TextEditingController _controller = TextEditingController();

  void _addTodo() {
    setState(() {
      _todos.add(_controller.text);
      _controller.clear();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('To-Do List')),
      body: Column(
        children: [
          TextField(controller: _controller, decoration: const
InputDecoration(labelText: 'New To-Do')),
          ElevatedButton(onPressed: _addTodo, child: const Text('Add')),
          Expanded(
            child: ListView.builder(
              itemCount: _todos.length,
              itemBuilder: (context, index) => ListTile(title:
Text(_todos[index])),
            ),
          ),
        ],
      ),
```

```
    );
  }
}
```

3. **Basic API Call с Future (для GitHub, из Section 4). Добавь http: ^0.13.5 в pubspec.yaml.**

```dart
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<List<dynamic>> fetchGitHubIssues(String token) async {
  final response = await http.get(
    Uri.parse('https://api.github.com/user/issues'), // Или твой repo
    headers: {'Authorization': 'token $token'},
  );
  if (response.statusCode == 200) {
    return json.decode(response.body);
  } else {
    throw Exception('Failed to load issues');
  }
}

// Usage in widget:
FutureBuilder<List<dynamic>>(
  future: fetchGitHubIssues('your_token'),
  builder: (context, snapshot) {
    if (snapshot.hasData) {
      return ListView.builder(
        itemCount: snapshot.data!.length,
        itemBuilder: (context, index) => ListTile(title:
Text(snapshot.data![index]['title'])),
      );
    } else if (snapshot.hasError) {
      return Text('Error: ${snapshot.error}');
    }
    return const CircularProgressIndicator();
  },
)
```

4. **Riverpod State (для global to-do, из Section 6). Добавь riverpod: ^2.0.0.**

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';

final todoProvider = StateNotifierProvider<TodoNotifier, List<String>>
((ref) => TodoNotifier());
```

```
class TodoNotifier extends StateNotifier<List<String>> {
  TodoNotifier() : super([]);

  void addTodo(String todo) {
    state = [...state, todo];
  }
}

// В widget:
Consumer(
  builder: (context, ref, child) {
    final todos = ref.watch(todoProvider);
    return ListView.builder(
      itemCount: todos.length,
      itemBuilder: (context, index) => Text(todos[index]),
    );
  },
)
```

Для OAuth: Используй oauth2 package — пример в доке pub.dev. Если нужно больше snippets (например, полный auth flow) или уточнения — скажи. Давай стартуем: Что первым — setup или конкретный snippet? 😊

Детали Riverpod в курсе

Андрей, в курсе **Flutter & Dart - The Complete Guide [2025 Edition]** (обновлён до 2026, но структура та же) Riverpod разбирается подробно, но **не в самом начале**, а в разделе про управление состоянием на уровне всего приложения. Это один из самых важных модулей для реальных проектов вроде твоего GitDolt, где нужно делить состояние (например, список to-do, статус авторизации) между экранами без пропсов drilling.

Где именно Riverpod в курсе

- **Основной раздел**: "Managing App-wide State [MEALS APP]"
  - Номер модуля примерно 177–192 (в зависимости от обновлений, но это ~16 лекций, ~1 час 12 мин).
  - Это часть проекта Meals App (приложение с рецептами еды, где есть избранное, фильтры и т.д.).
  - Автор переходит на Riverpod после обсуждения проблем с локальным state (setState) и Provider (старый пакет).

Полный список лекций в этом разделе (из syllabus курса):

- Module Introduction
- What’s The Problem? (почему нужен app-wide state)
- Installing the Solution: Riverpod (установка пакета flutter_riverpod)
- How State Management with Riverpod Works (основные концепции: providers, ref, watch/read)
- Creating a Provider (простой Provider для read-only данных)
- Using a Provider (как читать в UI с ConsumerWidget или ref.watch)
- Creating a More Complex Provider with StateNotifier (StateNotifierProvider для mutable state)
- Using the FavoritesProvider (пример с избранными рецептами)
- Triggering a Notifier Method (как вызывать методы notifier'а для обновлений)
- Getting Started with Another Provider (дополнительный provider)
- Combining Local & Provider-managed State (смешивание setState и Riverpod)
- Outsourcing State Into The Provider (перенос всего состояния в provider)
- Connecting Multiple Providers With Each Other (Dependent Providers) (зависимые провайдеры, ref.watch внутри другого)
- Swapping The Favorite Button Based On Provider State (динамический UI на основе состояния)
- Module Summary
- riverpod vs provider – There are many Alternatives! (сравнение с Provider, почему Riverpod лучше)

Riverpod также мелькает позже в challenge solutions (например, в проекте с местами/картами: "Adding riverpod & A Provider", "Adding Places with Provider & Displaying Places" — там применяют в практике).

Почему это идеально для GitDoIt

- В Meals App ты увидишь почти то же, что нужно тебе: список элементов (рецепты → твои to-do/issues), добавление/удаление, фильтры (можно добавить completed/uncompleted), избранное (похоже на sync с GitHub).
- Основные типы, которые использует курс:
  - **Provider** — для простых значений (например, текущий user после auth).
  - **StateNotifierProvider** — для сложного состояния (список to-do, методы add/remove/sync).

- **ref.watch / ref.read** — для чтения в UI.
- Dependent providers — если один провайдер зависит от другого (например, список issues зависит от token auth).

Курс использует Riverpod **2.x** (актуальный на 2025–2026), с StateNotifier (в Riverpod 2+ это legacy, но всё ещё работает отлично; новые AsyncNotifier для async — курс не углубляется, но базово хватит).

Как вплести в твой план (обновлённый под Riverpod)

Ты стартуешь сегодня (18 Jan 2026). Добавь этот раздел именно в дни 9–11 (Sync Logic & State):

- **Дни 9–11 (26–28 Jan): Sync Logic & State**
  - Курс: Пройди полностью "Managing App-wide State [MEALS APP]" (лекции 177–192).
  - GitDoIt задачи:
    - Установи `flutter_riverpod: ^2.5.1` (или latest) в pubspec.yaml.
    - Создай providers.dart файл:
      - authProvider (StateNotifier для token/user).
      - todosProvider (StateNotifier для списка to-do, с методами syncToGitHub).
    - Перепиши ToDoScreen на ConsumerWidget + ref.watch(todosProvider).
    - Добавь notifier методы: addTodo (push как issue на GitHub), toggleComplete.
    - Протестируй: изменения в одном месте обновляют UI везде.

Если забыл основы — перед этим быстро refresh Sections 5–6 (Multi-Screen + базовый state), но Riverpod объясняется с нуля в этом модуле.

Быстрый refresh snippets (на основе курса + адаптация под GitDoIt)

1. **Setup Riverpod в main.dart** (обязательно оберни MaterialApp в ProviderScope):

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

void main() {
  runApp(const ProviderScope(child: MyApp()));
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'GitDoIt',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const LoginScreen(), // или Home
    );
  }
}
```

2. **Пример todosProvider (StateNotifier):**

```Dart
import 'package:flutter_riverpod/flutter_riverpod.dart';

// Модель
class Todo {
  final String id; // или issue number от GitHub
  final String title;
  final bool completed;

  Todo({required this.id, required this.title, this.completed = false});

  Todo copyWith({bool? completed}) => Todo(
        id: id,
        title: title,
        completed: completed ?? this.completed,
      );
}

// Notifier
class TodosNotifier extends StateNotifier<List<Todo>> {
  TodosNotifier() : super([]);

  void addTodo(String title) {
    // Здесь можно добавить API call на GitHub create issue
    state = [...state, Todo(id: DateTime.now().toString(), title: title)];
  }

  void toggleTodo(String id) {
    state = [
      for (final todo in state)
        if (todo.id == id) todo.copyWith(completed: !todo.completed) else
todo,
    ];
  }
```

```
  // Future метод для sync (async)
  Future<void> syncWithGitHub() async {
    // TODO: pull issues from GitHub и update state
  }
}

final todosProvider = StateNotifierProvider<TodosNotifier, List<Todo>>
((ref) => TodosNotifier());
```

3. **Использование в UI** (ToDoScreen как ConsumerWidget):

```Dart
class ToDoScreen extends ConsumerWidget {
  const ToDoScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final todos = ref.watch(todosProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('GitDoIt To-Do')),
      body: ListView.builder(
        itemCount: todos.length,
        itemBuilder: (context, index) {
          final todo = todos[index];
          return ListTile(
            title: Text(todo.title),
            leading: Checkbox(
              value: todo.completed,
              onChanged: (_) =>
ref.read(todosProvider.notifier).toggleTodo(todo.id),
            ),
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          // Добавь input dialog, затем
ref.read(todosProvider.notifier).addTodo(text);
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Это прям из стиля курса — простой, declarative, testable. После прохождения модуля Meals App ты сможешь легко адаптировать под auth + GitHub sync (добавь authProvider, и в todosProvider используй ref.watch(authProvider) для token).

Если хочешь — давай уточним: пройтишь этот раздел первым? Или нужно больше snippets для auth + Riverpod? Или адаптировать Logseq под этот модуль? Погружаемся дальше! 😊

Спасибо. Подумай ещё раз. Возьми задачи из нашей беседы где мы разбили по неделям (5 недель) соотнести их с курсом ещё раз и напиши ещё более подробный план со сниппетами, и другими добавлениями которые ты сейчас предложил. Помоги организовать подход чтобы получить новые знания в тайм / проджек менеджменте, в современных подходах. Очень подробный план. Я не боюсь много текста. Сейчас я переезжаю из одной квартиры а другую. Я начал проходить курс и свой проект, но сегодня и завтра я не могу физически ими заниматься. По этому я хочу как можно лучше подготовиться для того чтоб б можно было без интернета в том числе (курс могу смотреть с телефона) заниматься этим проектом. В новой квартире пока нет интернета, но это мне может помочь, избавит от возможных сосков на посмотреть Ютуб . Я пытаюсь бороться с поокоасттнацтей .

Размышление на протяжении 21s

Подробный 5-Недельный План для Проекта GitDoIt: Интеграция с Курсом Udemy, Time/Project Management и Оффлайн-Подготовка

Андрей, спасибо за детали — это поможет сделать план максимально полезным. Я подумал заново, опираясь на нашу историю бесед (включая ранний breakdown на 5 недель, где мы расширили от MVP к полному проекту с advanced features, чтобы глубже освоить Flutter). В той беседе мы разделили на 5 недель для комфортного темпа обучения: Week 1 — основы и setup; Week 2 — UI и interactivity; Week 3 — state и navigation; Week 4 — API/sync; Week 5 — testing/deploy/polish. Я адаптировал это под твой текущий контекст: переезд (стартуем с 20 января 2026, чтобы дать буфер на 18-19), борьбу с прокрастинацией (маленькие шаги, rewards), оффлайн-режим (подготовка материалов) и фокус на новых знаниях в time/project management.

Общий Обзор Плана:

- **Продолжительность:** 5 недель (20 Jan — 23 Feb 2026), с MVP готовым к концу Week 3 (для мотивации), а остальное — polish и advanced.
- **Ежедневная Структура:** 2-4 часа/день (time-boxing, чтобы не burnout). Утро: 10-min self-standup (что вчера, сегодня, blockers). День: Pomodoro (25 min work + 5 break). Вечер: 5-min journal (что узнал, commit).
- **Цели:** MVP (auth, basic to-do sync с GitHub issues) к 5 Feb. Полный проект (с offline, animations, deploy) к 23 Feb. Освоить Flutter + новые PM техники.
- **Соотнесение с Курсом:** Курс "Flutter & Dart - The Complete Guide [2025 Edition]" (~30ч) разбит параллельно. Фокус на Sections 1-7 для core, потом advanced (8-12 для animations, storage, deploy). После каждой section — применяй в GitDoIt.
- **Time/Project Management (Современные Подходы):**
  - **Solo Agile (из Shape Up/Basecamp):** Короткие циклы (недели как "shapes"), фокус на betting (выбери 3-5 задач/неделю), cool-down (выходные на rest).
  - **Kanban в Logseq:** Доска с колонками TODO/Doing/Done. WIP limit: 2 задачи max.
  - **Anti-Procrastination:** Start small (первые 5 min — просто открой VS Code). Rewards: После сессии — кофе/прогулка. Track streaks в Logseq.
  - **Retrospectives:** Еженедельно (воскресенье): What went well? What to improve? Adjust plan.
  - **Tools:** Logseq (оффлайн), VS Code (с Flutter extension, offline после setup), GitHub Desktop (для commits без net).
- **Оффлайн-Подготовка:** Сегодня-завтра (18-19 Jan) — скачай всё. Курс: Загрузи видео в Udemy app на телефон (оффлайн viewing). Snippets: Скопируй в Markdown файл в Logseq или txt. Flutter: Установи SDK заранее, скачай pub packages (flutter pub get --offline). Doc: Скачай flutter.dev docs via `flutter doctor --android-licenses` + offline PDFs. GitDoIt repo: Clone локально, работай без push до интернета.
- **Мотивация:** Борьба с прокрастинацией — фокус на learning wins (каждый день note "Что нового узнал по Dart/PM"). Если slip — no guilt, restart next day.

Теперь — **подробный план по неделям**. Каждый: Курс sections, GitDoIt tasks, snippets, PM tips. Используй Logseq template в конце для tracking.

Подготовка (18-19 Jan 2026: Переезд Days)

- **Цель:** Setup offline environment, чтобы с 20 Jan стартовать без интернета.

- **Задачи:**
  - Скачай Udemy курс видео (Sections 1-12) на телефон.
  - Установи Flutter SDK (если не), VS Code + extensions (Flutter, Dart). Run `flutter doctor` для check.
  - Create GitDoIt repo локально (git init), добавь .gitignore для Flutter.
  - Скопируй все snippets ниже в Logseq page [[GitDoIt Snippets]] — printable для offline.
  - PM Learning: Прочитай/скачай статьи (browse offline): "Shape Up" by Basecamp (PDF), "Pomodoro Technique" guide. Note: Solo Agile — bet on weekly goals, не micromanage.
- **Time Estimate:** 1-2 часа total (сегодня/завтра snippets).
- **Anti-Procrastination:** Start with 5-min task: Just open Udemy app.

Week 1: Setup & Dart/Flutter Basics (20-26 Jan 2026)

- **Цель:** Refresh fundamentals, build empty app. Learn: Basic PM (time-boxing).
- **Курс Соотнесение:** Sections 1-2 (~5ч: Introduction + Basics I, Roll Dice App). Освежи Dart (vars, functions, classes).
- **Daily Breakdown:**
  - **Day 1 (20 Jan):** Курс: Section 1. GitDoIt: Install, create project. Snippet: Basic main.dart.
  - **Day 2 (21 Jan):** Курс: Start Section 2. GitDoIt: Add Scaffold, run on emulator.
  - **Day 3-4 (22-23 Jan):** Курс: Finish Section 2. GitDoIt: Basic stateless widgets (Text, Button).
  - **Day 5 (24 Jan):** Practice: Build simple counter app (stateful). Commit.
  - **Weekend (25-26 Jan):** Light review + retrospective: Adjust if slow.
- **GitDoIt Tasks:** Setup repo, basic UI skeleton (login button stub).
- **Snippets:**
  - Basic Setup (main.dart):

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
```

```
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'GitDoIt',
      home: Scaffold(appBar: AppBar(title: const Text('Welcome')),
body: const Center(child: Text('Start Learning'))),
    );
  }
}
```

- Simple Stateful Counter (для refresh state):

```dart
class CounterScreen extends StatefulWidget {
  const CounterScreen({super.key});
  @override
  _CounterScreenState createState() => _CounterScreenState();
}

class _CounterScreenState extends State<CounterScreen> {
  int _counter = 0;
  void _increment() => setState(() => _counter++);
  @override
  Widget build(BuildContext context) {
    return Column(children: [Text('$_counter'),
ElevatedButton(onPressed: _increment, child: const Text('+'))]);
  }
}
```

- **PM Tips:** Use Pomodoro: 4 sessions/day. Logseq Kanban: Create board [[GitDoIt Week1]] with TODO: Setup, Doing: Курс, Done: Commit. New Knowledge: Time-boxing prevents overwhelm — set alarm for end.

Week 2: UI Interactivity & Widgets (27 Jan - 2 Feb 2026)

- **Цель:** Build core UI (login, list). Learn: Kanban flow.
- **Курс Соотнесение:** Sections 3-4 (~10ч: Basics II + Interactivity, Quiz/Expense Tracker). Фокус: Lists, inputs, conditional UI.
- **Daily Breakdown:**
  - **Day 1-2 (27-28 Jan):** Курс: Section 3 (Quiz App). GitDoIt: Login screen with TextField.
  - **Day 3-4 (29-30 Jan):** Курс: Start Section 4. GitDoIt: To-do ListView, add item.
  - **Day 5 (31 Jan):** Курс: Finish Section 4. GitDoIt: Theming (dark mode).

    - **Weekend (1-2 Feb):** UI polish + retro: Measure progress (e.g., 80% UI done?).
- **GitDoIt Tasks:** Interactive to-do list (add/delete, no sync yet).
- **Snippets:**
    - Login Screen with Input:

```dart
class LoginScreen extends StatelessWidget {
  final TextEditingController _controller = TextEditingController();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(children: [
        TextField(controller: _controller, decoration: const
InputDecoration(labelText: 'GitHub Token')),
        ElevatedButton(onPressed: () {/* Stub auth */}, child: const
Text('Login')),
      ]),
    );
  }
}
```

    - To-Do ListView:

```dart
class ToDoList extends StatefulWidget {
  @override
  _ToDoListState createState() => _ToDoListState();
}

class _ToDoListState extends State<ToDoList> {
  List<String> _todos = [];
  void _add(String text) => setState(() => _todos.add(text));
  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: _todos.length,
      itemBuilder: (ctx, idx) => ListTile(title: Text(_todos[idx])),
    );
  }
}
```

- **PM Tips:** Kanban WIP: Limit 2 tasks (e.g., UI + Курс). Anti-Procr: Reward after day — watch saved video (не YouTube). New: Flow state — chain small wins to build momentum.

Week 3: State Management & Navigation (3-9 Feb 2026)

- **Цель:** Integrate Riverpod, navigation. MVP ready. Learn: Retrospectives.
- **Курс Соотнесение:** Sections 5-6 (~5ч: Multi-Screen + App-wide State, Meals App). Deep dive Riverpod.
- **Daily Breakdown:**
  - **Day 1-2 (3-4 Feb):** Курс: Section 5 (Navigation). GitDoIt: Routes (login -> todo).
  - **Day 3-4 (5-6 Feb):** Курс: Section 6 (Riverpod). GitDoIt: Migrate state to Riverpod.
  - **Day 5 (7 Feb):** Practice: Dependent providers. Commit MVP.
  - **Weekend (8-9 Feb):** Retro: Analyze delays, adjust.
- **GitDoIt Tasks:** App-wide state for todos, navigation flow.
- **Snippets:** (Расширенные для Riverpod, как в курсе Meals App)
  - ProviderScope in main.dart (уже был, repeat).
  - Auth Provider:

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';

class AuthNotifier extends StateNotifier<String?> {
  AuthNotifier() : super(null);
  void login(String token) => state = token;
}

final authProvider = StateNotifierProvider<AuthNotifier, String?>
((ref) => AuthNotifier());
```

  - Todos with Dependency:

```dart
final todosProvider = StateNotifierProvider<TodosNotifier,
List<Todo>>((ref) {
  final token = ref.watch(authProvider);
  return TodosNotifier(token); // Use token for API later
});

class TodosNotifier extends StateNotifier<List<Todo>> {
  final String? token;
  TodosNotifier(this.token) : super([]);
```

```
  // Methods as before
}
```

- Navigation with Riverpod:

```dart
Navigator.push(context, MaterialPageRoute(builder: (_) => const ToDoScreen()));
// In ToDoScreen: Use ConsumerWidget + ref
```

- **PM Tips:** Weekly Retro: Journal "Wins: Learned Riverpod. Improve: More breaks." New: Outcome-based goals (не "do task", а "achieve MVP").

Week 4: API Integration & Sync (10-16 Feb 2026)

- **Цель:** Add GitHub API, async sync. Learn: Dependency mapping.
- **Курс Соотнесение:** Section 4 revisit (Futures) + Advanced Networking (если в курсе, ~3ч).
- **Daily Breakdown:**
  - **Day 1-2 (10-11 Feb):** Курс: Async/Futures. GitDoIt: Basic HTTP.
  - **Day 3-4 (12-13 Feb):** GitDoIt: OAuth setup, fetch issues.
  - **Day 5 (14 Feb):** Sync logic in Riverpod. Test.
  - **Weekend (15-16 Feb):** Bug fix + retro.
- **GitDoIt Tasks:** Full sync (pull/push issues as todos).
- **Snippets:**
  - OAuth Setup (add oauth2 package offline via pub get):

```dart
import 'package:oauth2/oauth2.dart' as oauth2;

Future<void> authenticate() async {
  final client = await oauth2.clientCredentialsGrant(
    Uri.parse('https://github.com/login/oauth/access_token'),
    'client_id', 'client_secret' // Replace with yours
  );
  // Use client.credentials.accessToken for API
}
```

  - Fetch Issues in Notifier:

```dart
```

```
Future<void> sync() async {
  if (token == null) return;
  final response = await
http.get(Uri.parse('https://api.github.com/user/issues'), headers:
{'Authorization': 'token $token'});
  final data = json.decode(response.body);
  state = data.map((item) => Todo(id: item['id'].toString(), title:
item['title'])).toList();
}
```

- **PM Tips:** Dependency Graph: In Logseq, draw links (e.g., API depends on State). New: Critical Path — prioritize sync for MVP.

Week 5: Testing, Polish & Deployment (17-23 Feb 2026)

- **Цель:** Finalize, deploy. Learn: Full cycle review.
- **Курс Соотнесение:** Sections 7-9 (~5ч: Internals, Testing, Animations).
- **Daily Breakdown:**
  - **Day 1-2 (17-18 Feb):** Курс: Section 7-8. GitDoIt: Unit tests.
  - **Day 3-4 (19-20 Feb):** Animations, offline (Hive package).
  - **Day 5 (21 Feb):** Deploy APK, final commit.
  - **Weekend (22-23 Feb):** Full retro, celebrate.
- **GitDoIt Tasks:** Tests, animations, build for Android/iOS.
- **Snippets:**
  - Unit Test:

```dart
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('Add Todo', () {
    final notifier = TodosNotifier(null);
    notifier.addTodo('Test');
    expect(notifier.state.length, 1);
  });
}
```

  - Animation (Hero for transitions):

```dart
Hero(tag: 'todo_${todo.id}', child: ListTile(title:
```

```
Text(todo.title)));
```

- **PM Tips:** End Retrospective: Overall learnings. New: Iterative Improvement — use this project as template for future.

Logseq Template for Entire Plan

Создай page [[GitDoIt 5-Week Plan]]. Paste this:

```text
- # GitDoIt Kanban
  - TODO Week 1 Tasks #week1
    DEADLINE: <2026-01-26 Sun>
    - Sub: Курс Section 1
    - Sub: Setup project
  - DOING Current #week1
  - DONE Completed #week1

- Weekly Retro Template
  - Wins:
  - Improves:
  - Adjustments:

- Snippets Block
  - [[Basic Setup]] :code: ```dart ... ```
  - ... (add all)

- Journal Daily
  - [[2026-01-20]] What did: ... Learned: ... Blockers: ...
```

Это полный, actionable план — используй его offline. Если нужно доработки (e.g., more snippets) — скажи после переезда. Удачи с переездом, держись! 😊

Можно ещё подробнее. С объяснением терминов. Я половину не понимаю. И можно на русском, но термины на английском (курс на английском, чтобы я точно понимал очки речь)

Подробный 5-Недельный План для Проекта GitDoIt: Интеграция с Курсом Udemy, Time Management и Project Management с Объяснениями Терминов

Андрей, спасибо, что уточнил — я вижу, что некоторые термины из предыдущего плана могут быть unfamiliar (незнакомыми), особенно если ты освежаешь знания по Flutter. Я переработал план, сделав его ещё более детальным: добавил объяснения для всех ключевых терминов при первом упоминании (с переводом

и простым описанием), разбил на меньшие шаги, добавил больше примеров и snippets (фрагментов кода). Всё на русском, но термины оставил на английском, как ты просил — чтобы они совпадали с курсом Udemy (который на английском) и ты точно понимал, о чём речь. Если что-то всё равно неясно, скажи, углубим дальше.

Общий Обзор Плана (с Объяснениями):

- **Продолжительность:** 5 недель, начиная с 20 января 2026 (чтобы дать тебе буфер на переезд 18-19 января). Каждая неделя — как "cycle" (цикл) в подходе Solo Agile (это упрощённая версия Agile-методологии для одного человека: фокус на гибкости, коротких итерациях без лишней бюрократии, как в командах; помогает не растягивать проект).
- **Ежедневная Структура:** 2-4 часа в день, чтобы избежать burnout (выгорания — когда от перегрузки теряешь мотивацию). Используй Pomodoro Technique (техника Помодоро: работай 25 минут сосредоточенно, потом 5 минут перерыва; помогает бороться с procrastination — прокрастинацией, т.е. откладыванием дел). Утро: 10-минутный self-standup (само-стендап: мысленно или в Logseq ответь "Что сделал вчера? Что сегодня? Какие blockers — препятствия?"). Вечер: 5-минутный journal (журнал: запиши, что узнал и commit — зафиксировал в git).
- **Цели:** MVP (Minimum Viable Product — минимально жизнеспособный продукт: базовая версия твоего GitDoIt с авторизацией и sync to-do с GitHub issues) к концу 3-й недели. Полный проект (с доп. features как offline support — поддержка без интернета) к 23 февраля. Это поможет мотивации: быстрые wins (победы).
- **Соотнесение с Курсом:** Курс "Flutter & Dart - The Complete Guide [2025 Edition]" разбит параллельно. Каждая section (секция — модуль курса) применяется сразу в GitDoIt. Курс использует проекты вроде Roll Dice App (приложение с кубиком) для практики.
- **Time Management и Project Management (с Объяснениями):**
  - Time Management: Фокус на time-boxing (ограничение времени: фиксируй слот, напр. "2 часа на UI", и стоп по таймеру; предотвращает бесконечность).
  - Project Management: Используй Kanban (метод визуализации задач: доска с колонками TODO — к выполнению, Doing — в работе, Done — сделано; WIP limit — лимит на одновременные задачи, напр. не больше 2, чтобы не распыляться). Retrospectives (ретроспективы: еженедельный анализ "Что

пошло хорошо? Что улучшить?"). Anti-Procrastination: Start small (начни с малого: первые 5 мин — просто открой VS Code). Rewards (награды: после сессии — чай или прогулка).

  - Tools: Logseq (оффлайн-ноутбук для outlining — структурирования идей в блоки; используй для Kanban). VS Code (редактор кода с extension для Flutter). Git (система контроля версий: commit — сохраняй изменения с описанием).

- **Оффлайн-Подготовка:** Скачай курс в Udemy app, snippets в текст-файл, Flutter SDK и packages (run `flutter pub get --offline` — скачай зависимости без net). Работай локально, push в GitHub позже.
- **Структура Плана:** Каждая неделя — цели, курс, daily breakdown (ежедневный разбор), tasks (задачи), snippets, PM tips (советы по менеджменту). Добавил объяснения терминов в скобках.

Подготовка (18-19 января 2026: Дни Переезда)

- **Цель:** Подготовить всё offline, чтобы с 20 января не зависеть от интернета. Это поможет в борьбе с procrastination — всё готово, просто садись и работай.
- **Задачи (Подробно):**
  - Скачай видео курса (Sections 1-12) в Udemy app на телефон (оффлайн-viewing: смотри без net).
  - Установи Flutter SDK (software development kit — набор инструментов для Flutter; скачай с flutter.dev, run `flutter doctor` — проверь установку).
  - Создай локальный repo (repository — хранилище кода) для GitDoIt: В терминале `git init`, добавь .gitignore (файл, игнорирующий ненужные файлы, как build-артефакты).
  - Скопируй все snippets ниже в Logseq или txt-файл (назови "GitDoIt_Snippets.md").
  - PM Learning: Скачай PDF "Shape Up" by Basecamp (гибкий метод для инди-проектов: "betting" — выбор ключевых задач на цикл). Прочитай intro по Pomodoro.
- **Time Estimate:** 1-2 часа всего (разбей на Pomodoro).
- **Anti-Procrastination Tip:** Начни с самого простого: Открой Udemy и скачай одну section. Reward: Переездный кофе.
- **New Knowledge Goal:** Пойми, как offline-setup снижает barriers (барьеры) для старта.

Week 1: Setup & Dart/Flutter Basics (20-26 января 2026)

- **Цель:** Освежить основы Dart (язык программирования для Flutter) и Flutter (framework — каркас для создания мобильных apps). Построй пустой app. Learn: Как time-boxing помогает фокусу.
- **Курс Соотнесение:** Sections 1-2 (~5 часов: Introduction — введение, Basics I — основы, проект Roll Dice App — приложение с кубиком для рандома).
- **Daily Breakdown (Ежедневный Разбор, с Шагами):**
  - **Day 1 (20 Jan):** Курс: Section 1 (установка, первый run). GitDoIt: Создай проект (`flutter create gitdoit`), run на emulator (эмулятор — виртуальное устройство для теста).
  - **Day 2 (21 Jan):** Курс: Начало Section 2 (variables — переменные, functions — функции). GitDoIt: Добавь Scaffold (основная структура экрана: appBar — верхняя панель, body — тело).
  - **Day 3 (22 Jan):** Курс: Stateful vs Stateless Widgets (stateful — с изменяемым состоянием, stateless — статичный). GitDoIt: Добавь простой Text widget.
  - **Day 4 (23 Jan):** Курс: Заверши Section 2. Practice: Построй counter (счётчик) для refresh.
  - **Day 5 (24 Jan):** GitDoIt: Commit changes. Light practice.
  - **Weekend (25-26 Jan):** Retrospective: Запиши в Logseq "Wins: Освежил Dart. Improves: Больше Pomodoro".
- **GitDoIt Tasks (Подробно):** Setup repo, базовый UI skeleton (скелет — основа: экран с заголовком).
- **Snippets (Фрагменты Кода, с Объяснениями):**
  - Basic Setup (main.dart — главный файл запуска app):

```dart
import 'package:flutter/material.dart'; // Импорт Material Design (стиль UI от Google).

void main() => runApp(const MyApp()); // Запуск app.

class MyApp extends StatelessWidget { // StatelessWidget — виджет без состояния.
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) { // build — метод, строящий UI.
    return MaterialApp( // Основной app контейнер.
```

```
      title: 'GitDoIt',
      home: Scaffold( // Scaffold — базовая структура экрана.
        appBar: AppBar(title: const Text('Welcome')), // AppBar — верхняя бар.
        body: const Center(child: Text('Start Learning')), // Body — основное содержание.
      ),
    );
  }
}
```

Объяснение: Это создаёт пустой экран. Run: `flutter run` в терминале.

- Simple Stateful Counter (для понимания state — состояния):

```Dart
class CounterScreen extends StatefulWidget { // StatefulWidget — с изменяемым state.
  const CounterScreen({super.key});
  @override
  _CounterScreenState createState() => _CounterScreenState(); // createState — создаёт state.
}

class _CounterScreenState extends State<CounterScreen> { // State класс.
  int _counter = 0; // Private variable (_ — приватная).
  void _increment() => setState(() => _counter++); // setState — обновляет UI.
  @override
  Widget build(BuildContext context) {
    return Column(children: [Text('$_counter'), ElevatedButton(onPressed: _increment, child: const Text('+'))]);
  }
}
```

Объяснение: Нажимаешь кнопку — счётчик растёт, UI обновляется.

- **PM Tips (Советы по Менеджменту):** Kanban в Logseq: Создай блоки TODO/Doing/Done. WIP limit: 2 задачи. New Knowledge: Time-boxing — как "ящик времени", фиксирует фокус, борется с perfectionism (перфекционизмом — желанием сделать идеально, что тормозит).

Week 2: UI Interactivity & Widgets (27 января - 2 февраля 2026)

- **Цель:** Построй interactive UI (интерактивный интерфейс: кнопки, inputs). Learn: Kanban flow (поток — как задачи двигаются по доске).

- **Курс Соотнесение:** Sections 3-4 (~10 часов: Basics II — основы 2, Interactivity — интерактивность, проекты Quiz App — викторина, Expense Tracker — трекер расходов).
- **Daily Breakdown:**
  - **Day 1 (27 Jan):** Курс: Section 3 (conditional rendering — условное отображение). GitDoIt: Login screen с TextField (поле ввода).
  - **Day 2 (28 Jan):** Курс: Lists и mapping (отображение данных в список). GitDoIt: Добавь ListView (список виджетов).
  - **Day 3 (29 Jan):** Курс: Input handling (обработка ввода). GitDoIt: Кнопка add to-do.
  - **Day 4 (30 Jan):** Курс: Theming (темы: colors, fonts). GitDoIt: Добавь dark mode.
  - **Day 5 (31 Jan):** Practice: Fix bugs (ошибки). Commit.
  - **Weekend (1-2 Feb):** Retrospective: "Wins: UI работает. Improves: Больше тестов на emulator".
- **GitDoIt Tasks:** Interactive to-do list (add/delete items локально, без sync).
- **Snippets:**
  - Login Screen with Input:

```dart
class LoginScreen extends StatelessWidget {
  final TextEditingController _controller = TextEditingController();
// Controller — управляет текстом в поле.
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(children: [
        TextField( // TextField — поле ввода.
          controller: _controller,
          decoration: const InputDecoration(labelText: 'GitHub
Token'), // Decoration — оформление.
        ),
        ElevatedButton(onPressed: () { /* Stub — заглушка для auth
*/ }, child: const Text('Login')),
      ]),
    );
  }
}
```

Объяснение: Вводишь token (токен — ключ доступа), нажимаешь — stub (заглушка: временный код).

- To-Do ListView with Add:

```dart
class ToDoList extends StatefulWidget {
  @override
  _ToDoListState createState() => _ToDoListState();
}

class _ToDoListState extends State<ToDoList> {
  List<String> _todos = []; // List — массив данных.
  final TextEditingController _controller = TextEditingController();
  void _add() {
    setState(() => _todos.add(_controller.text)); // Add — добавить
в список.
    _controller.clear(); // Clear — очистить поле.
  }
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      TextField(controller: _controller),
      ElevatedButton(onPressed: _add, child: const Text('Add')),
      ListView.builder( // Builder — строит список динамически.
        itemCount: _todos.length,
        itemBuilder: (ctx, idx) => ListTile(title:
Text(_todos[idx])),
      ),
    ]);
  }
}
```

Объяснение: Добавляешь текст — появляется в списке.

- **PM Tips:** Kanban Flow: Двигай задачи из TODO в Done. Anti-Procr: Если stuck (застрял) — break и вернись. New: Visual progress (визуальный прогресс) мотивирует.

Week 3: State Management & Navigation (3-9 февраля 2026)

- **Цель:** Добавь Riverpod (библиотека для state management — управления состоянием: делит данные между экранами без prop drilling — передачи через цепочку). Navigation (навигация: переходы между экранами). MVP ready.
- **Курс Соотнесение:** Sections 5-6 (~5 часов: Multi-Screen Apps — многоэкранные apps, App-wide State — состояние всего app, проект Meals App — приложение с рецептами).

- **Daily Breakdown:**
  - **Day 1 (3 Feb):** Курс: Section 5 (Navigator — инструмент навигации). GitDoIt: Добавь route (маршрут: от login к to-do).
  - **Day 2 (4 Feb):** Курс: Passing data (передача данных между экранами). GitDoIt: Pass token.
  - **Day 3 (5 Feb):** Курс: Section 6 (Riverpod intro). GitDoIt: Установи package (пакет: `flutter_riverpod`).
  - **Day 4 (6 Feb):** Курс: StateNotifierProvider (провайдер для mutable state — изменяемого состояния). GitDoIt: Migrate (перенеси) todos в Riverpod.
  - **Day 5 (7 Feb):** Practice: Dependent providers (зависимые: один от другого). Commit MVP.
  - **Weekend (8-9 Feb):** Retrospective: "Wins: Riverpod работает. Improves: Async в sync".
- **GitDoIt Tasks:** App-wide state, полный flow (поток: login → list).
- **Snippets:**
  - ProviderScope (обёртка для Riverpod в main.dart):

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart'; // Импорт Riverpod.

void main() {
  runApp(const ProviderScope(child: MyApp())); // ProviderScope — контейнер для providers.
}
```

    Объяснение: Включает Riverpod для всего app.

  - Auth Provider (пример StateNotifier — notifier для уведомлений об изменениях):

```dart
class AuthNotifier extends StateNotifier<String?> { // StateNotifier — класс для управления state.
  AuthNotifier() : super(null); // super — вызов родителя.
  void login(String token) => state = token; // State — текущее значение.
}
```

```
final authProvider = StateNotifierProvider<AuthNotifier, String?>
((ref) => AuthNotifier()); // Provider — поставщик state.
```

Объяснение: Хранит token глобально.

- Todos Provider with Dependency:

```Dart
final todosProvider = StateNotifierProvider<TodosNotifier,
List<Todo>>((ref) { // ref — ссылка на другие providers.
  final token = ref.watch(authProvider); // watch — наблюдает за
изменениями.
  return TodosNotifier(token);
});

class TodosNotifier extends StateNotifier<List<Todo>> {
  final String? token;
  TodosNotifier(this.token) : super([]);
  void addTodo(String title) => state = [...state, Todo(id:
DateTime.now().toString(), title: title)];
}
```

Объяснение: Todos зависит от auth; add обновляет state.

- Usage in UI (ConsumerWidget — виджет, потребляющий state):

```Dart
class ToDoScreen extends ConsumerWidget { // Consumer — читает
providers.
  @override
  Widget build(BuildContext context, WidgetRef ref) { // WidgetRef —
ref для UI.
    final todos = ref.watch(todosProvider);
    return ListView.builder(
      itemCount: todos.length,
      itemBuilder: (ctx, idx) => ListTile(title:
Text(todos[idx].title)),
    );
  }
}
```

Объяснение: UI автоматически обновляется при изменении state.

- **PM Tips:** Retrospectives: Анализируй weekly. New: Outcome-based (ориентированные на результат: не "сделай код", а "достигни работающего MVP").

Week 4: API Integration & Sync (10-16 февраля 2026)

- **Цель:** Добавь API (Application Programming Interface — интерфейс для общения с GitHub). Async (асинхронный: futures — обещания будущих данных). Sync logic (логика синхронизации).
- **Курс Соотнесение:** Revisit Section 4 (Futures), advanced networking (~3 часа).
- **Daily Breakdown:**
  - **Day 1 (10 Feb):** Курс: Futures и async/await (ожидание асинхронных операций). GitDoIt: Добавь http package.
  - **Day 2 (11 Feb):** GitDoIt: OAuth (Open Authorization — протокол авторизации) setup.
  - **Day 3 (12 Feb):** GitDoIt: Fetch (получить) issues как todos.
  - **Day 4 (13 Feb):** GitDoIt: Push (отправить) new to-do как issue.
  - **Day 5 (14 Feb):** Test sync in Riverpod. Fix errors.
  - **Weekend (15-16 Feb):** Retrospective.
- **GitDoIt Tasks:** Полный sync (pull — тянуть данные, push — толкать).
- **Snippets:**
  - Basic API Call with Future:

```dart
import 'package:http/http.dart' as http; // http — пакет для запросов.
import 'dart:convert'; // json — для парсинга данных.

Future<List<Todo>> fetchIssues(String token) async { // async — асинхронная функция.
  final response = await http.get( // await — ждёт завершения.
    Uri.parse('https://api.github.com/user/issues'),
    headers: {'Authorization': 'token $token'},
  );
  if (response.statusCode == 200) { // statusCode — код ответа.
    final data = json.decode(response.body); // decode — раскодировать JSON.
    return data.map((item) => Todo(id: item['id'].toString(), title: item['title'])).toList();
  } else {
    throw Exception('Error'); // throw — выбросить ошибку.
  }
}
```

Объяснение: Получает issues, преобразует в Todos.

- Sync in Notifier:

```dart
class TodosNotifier extends StateNotifier<List<Todo>> {
  // ... previous
  Future<void> sync() async {
    if (token == null) return;
    final issues = await fetchIssues(token);
    state = issues;
  }
}
// Usage: ref.read(todosProvider.notifier).sync(); // read — разовый вызов.
```

  Объяснение: Обновляет state из API.

- **PM Tips:** Dependency Mapping (карта зависимостей: в Logseq нарисуй arrows — стрелки, напр. API зависит от Riverpod). New: Critical Path (критический путь: приоритизируй sync как ключ к MVP).

Week 5: Testing, Polish & Deployment (17-23 февраля 2026)

- **Цель:** Тесты, polish (шлифовка: улучшения), deploy (развёртывание: build APK — файл для Android).
- **Курс Соотнесение:** Sections 7-9 (~5 часов: Internals — внутренности Flutter, Testing — тесты, Animations — анимации).
- **Daily Breakdown:**
  - **Day 1 (17 Feb):** Курс: Section 7 (lifecycle — цикл жизни виджетов). GitDoIt: Добавь unit tests (юнит-тесты: проверка частей кода).
  - **Day 2 (18 Feb):** Курс: Testing. GitDoIt: Test sync.
  - **Day 3 (19 Feb):** Курс: Animations. GitDoIt: Добавь Hero (анимация переходов).
  - **Day 4 (20 Feb):** GitDoIt: Offline support (используй Hive — локальное хранилище).
  - **Day 5 (21 Feb):** Build APK (`flutter build apk`), test on device.
  - **Weekend (22-23 Feb):** Full Retrospective: Celebrate completion!
- **GitDoIt Tasks:** Тесты, анимации, final build.
- **Snippets:**
  - Unit Test:

```dart
import 'package:flutter_test/flutter_test.dart'; // flutter_test — пакет для тестов.

void main() {
  test('Add Todo Test', () { // test — функция теста.
    final notifier = TodosNotifier(null);
    notifier.addTodo('Test');
    expect(notifier.state.length, 1); // expect — проверка ожидания.
  });
}
```

Объяснение: Run: `flutter test` — проверяет логику.

- Hero Animation:

```dart
Hero( // Hero — анимация между экранами.
  tag: 'todo_${todo.id}', // tag — уникальный ID.
  child: ListTile(title: Text(todo.title)),
);
```

Объяснение: Плавный transition (переход) при клике.

- **PM Tips:** Full Cycle Review (обзор цикла: подведи итоги всего проекта). New: Iterative Improvement (итеративное улучшение: используй этот план как template — шаблон для будущих проектов).

Это максимально подробный план — с объяснениями, чтобы ты не путался. Если нужно добавить/изменить (напр. больше snippets по OAuth), или углубить термин — дай знать. Удачи с проектом и переездом! 😊

Я не вижу тут составляющую для создания контента для соц сетей, для продвижения приложения и просто моего документирования / журналирлвания (и возможной монетизации). Добавь это в такой прекрасный план. Это тоже отдельные часы на создание . Чтобы не забывать об этом. Мы это обсуждали тоже в той беседе. Проверь ещё может. Быть мы обсуждали там, но. Тут это не отображено.

Размышление на протяжении 22s

Подробный 5-Недельный План для Проекта GitDoIt: Интеграция с Курсом Udemy, Time Management, Project Management, Content Creation, Promotion, Journaling и Monetization (с Объяснениями Терминов)

Андрей, спасибо за напоминание — да, мы действительно обсуждали это в предыдущей беседе (я проверил историю: там шла речь о journaling в Logseq для

фиксации прогресса, создании контента для X (твой @berlogabob) для продвижения GitDolt, документировании процесса как open-source проекта и идеях монетизации, например, через GitHub Sponsors или Patreon после MVP). Ты прав, в предыдущем плане это не было отображено явно, так что я добавил новую компоненту: **Content Creation & Promotion** (создание контента и продвижение). Это будет отдельными часами (30-60 мин/день или неделю), чтобы не забывать и интегрировать в рутину.

Почему это важно и как вплести:

- Journaling (журналирование: ежедневные записи о прогрессе, learnings — уроках) поможет бороться с procrastination, фиксируя wins и мотивируя.
- Content для соцсетей (X, возможно LinkedIn или Habr): Посты с screenshots (скриншотами), code snippets, stories о разработке — для продвижения app (приложения) и личного бренда. Это может привести к feedback (отзывы) от community.
- Documentation (документирование: README в GitHub, notes в Logseq) — для open-source (открытый исходный код: публичный repo, где другие могут contribute — вносить вклад).
- Monetization (монетизация: способы заработка, напр. donations — пожертвования, или premium features) — подумаем в конце, как опцию (не обязательно, но как goal — цель).

Я добавил это в **ежедневную структуру** (15-30 мин journaling ежедневно, 30-60 мин content еженедельно) и в каждую неделю как отдельные tasks. Общее время на проект остаётся 2-4 часа/день, но с буфером на это (не перегружайся). Если интернета нет — journaling offline в Logseq, content drafts (черновики) в notes, post later.

Общий Обзор Плана (с Добавлениями и Объяснениями):

- **Продолжительность:** 5 недель (20 Jan — 23 Feb 2026), MVP к 9 Feb.
- **Ежедневная Структура (Обновлённая):**
  - Утро: 10-min self-standup.
  - Основная работа: 1.5-3 часа (Pomodoro на код/курс).
  - Вечер: 15-30 мин journaling (в Logseq: "Progress: ... Learnings: ... Content ideas: ...").
  - Еженедельно (воскресенье): 30-60 мин content creation (draft пост для X: текст + screenshot; идеи для promotion/monetization).

- **Цели:** MVP + полный app, плюс контент для visibility (видимости) в community.
- **Соотнесение с Курсом:** Как раньше.
- **Time/Project Management + Новое:** Добавь content в Kanban (колонка "Content TODO"). Anti-Procrastination: Связывай content с rewards (напр. "После поста — relax"). New Knowledge: Content marketing (контент-маркетинг: регулярные посты build audience — строят аудиторию).
- **Оффлайн-Подготовка:** Добавь drafts templates в Logseq (пример: "X Post: Week X — Learned about Riverpod, here's snippet...").

Подготовка (18-19 января 2026: Дни Переезда)

- **Цель:** Подготовить offline, включая content tools.
- **Задачи (Подробно + Добавления):**
  - Скачай курс, Flutter, etc. (как раньше).
  - В Logseq: Создай page [[GitDoIt Content]] с templates (шаблонами): Для journaling — "Date: Progress: Learnings: Content Idea:". Для X post — "Title: Text: Screenshot: Hashtags: #Flutter #Dart #IndieDev".
  - PM Learning: Скачай PDF о "Content Strategy for Devs" (если есть offline, или note ideas: regular posts для engagement — вовлечения).
  - Content Prep: Draft первый пост: "Starting GitDoIt: Learning Flutter for to-do app with GitHub sync. #LearningJourney".
- **Time Estimate:** +15 мин на content templates.
- **Anti-Procrastination Tip:** Small start: Запиши одну idea в Logseq.
- **New Knowledge Goal:** Journaling как habit (привычка) для accountability (ответственности).

Week 1: Setup & Dart/Flutter Basics (20-26 января 2026)

- **Цель:** Основы + start journaling/content.
- **Курс Соотнесение:** Sections 1-2.
- **Daily Breakdown:**
  - **Day 1-5:** Как раньше (setup, widgets).
  - **Ежедневно Добавлено:** 15-30 мин journaling: "Сегодня setup Flutter. Learned: Stateful vs Stateless. Idea for post: Share setup tips".
  - **Weekend Добавлено:** 30 мин content: Draft X post о старте проекта (screenshot empty app). Think promotion: "Share on X to get early feedback".

- **GitDoIt Tasks + Content:** Setup + document in README (GitHub file: описание проекта).
- **Snippets:** Как раньше.
- **PM Tips + Content:** Kanban: Добавь карточку "Content: Week1 Post". New: Documentation как часть promotion — хороший README attracts contributors.
- **Monetization Idea (Light):** Note: "После MVP — add GitHub Sponsors button in README".

Week 2: UI Interactivity & Widgets (27 января - 2 февраля 2026)

- **Цель:** UI + build content habit.
- **Курс Соотнесение:** Sections 3-4.
- **Daily Breakdown:**
  - **Day 1-5:** Как раньше (inputs, lists).
  - **Ежедневно Добавлено:** Journaling: "Built ListView. Challenges: Input handling. Content: Snippet for X".
  - **Weekend Добавлено:** 45 мин content: Create post "Week2: Building UI for GitDoIt. Here's to-do list snippet. #FlutterDev". Attach screenshot. Promotion: Tag #FlutterCommunity.
- **GitDoIt Tasks + Content:** UI + update README с UI description.
- **Snippets:** Как раньше + для content: Copy snippet в post draft.
- **PM Tips + Content:** Use journaling для content ideas (reuse learnings). New: Social proof (социальное доказательство: посты показывают expertise — экспертизу, helps monetization later).
- **Monetization Idea:** Journal: "Если app полезен, добавить premium sync features за donation".

Week 3: State Management & Navigation (3-9 февраля 2026)

- **Цель:** Riverpod + MVP, ramp up content.
- **Курс Соотнесение:** Sections 5-6.
- **Daily Breakdown:**
  - **Day 1-5:** Как раньше (navigation, Riverpod).
  - **Ежедневно Добавлено:** Journaling: "Migrated to Riverpod. Why better than setState? Content: Explain in post".

    - **Weekend Добавлено:** 60 мин content: Post "MVP ready! GitDoIt syncs to-do with GitHub. Code snippet: todosProvider. Feedback welcome! @berlogabob #OpenSource". Promotion: Share link to repo. Documentation: Full README with install guide.
- **GitDoIt Tasks + Content:** MVP + make repo public для sharing.
- **Snippets:** Как раньше + share Riverpod example in post.
- **PM Tips + Content:** Retrospectives: Include "Content wins: Got likes?". New: Thread in X (нить постов: series для storytelling — повествования о проекте).
- **Monetization Idea:** Add in README: "Support via GitHub Sponsors if you like the project".

Week 4: API Integration & Sync (10-16 февраля 2026)

- **Цель:** API + consistent content.
- **Курс Соотнесение:** Revisit Section 4.
- **Daily Breakdown:**
    - **Day 1-5:** Как раньше (OAuth, sync).
    - **Ежедневно Добавлено:** Journaling: "Added fetchIssues. Errors handled. Content: API tips for beginners".
    - **Weekend Добавлено:** 45 мин content: Post "Week4: Integrated GitHub API. Async magic! Snippet inside. #DartDev". Promotion: Ask for stars on GitHub.
- **GitDoIt Tasks + Content:** Sync + document API usage in Logseq/notes для future posts.
- **Snippets:** Как раньше + use in content.
- **PM Tips + Content:** Dependency Mapping: Include content как dependent task (зависимую: после код — post). New: Analytics (аналитика: track views on posts для motivation).
- **Monetization Idea:** Journal: "Monetize via tutorial series based on this (e.g., sell on Udemy)".

Week 5: Testing, Polish & Deployment (17-23 февраля 2026)

- **Цель:** Финал + monetization strategy.
- **Курс Соотнесение:** Sections 7-9.
- **Daily Breakdown:**
    - **Day 1-5:** Как раньше (tests, deploy).

    - **Ежедневно Добавлено:** Journaling: "Wrote unit tests. Deployed APK. Learnings: ... Content: Final wrap-up".
    - **Weekend Добавлено:** 60 мин content: Series finale post "GitDoIt complete! From idea to deploy. Full journey thread. Download here. #IndieApp". Promotion: Share on Habr/LinkedIn. Documentation: Complete project wiki in GitHub.
- **GitDoIt Tasks + Content:** Deploy + create promo video/screenshots для posts.
- **Snippets:** Как раньше + final test example in content.
- **PM Tips + Content:** Full Cycle Review: "Content impact: Gained followers?". New: Monetization planning (план: setup Patreon for dev tips based on journaling).
- **Monetization Focus:** Dedicate 30 мин: Setup GitHub Sponsors, think about blog/monetized content (e.g., "My Flutter Journey" as paid guide).

Logseq Template (Обновлённый с Content)

В [[GitDoIt 5-Week Plan]]:

```text
- # GitDoIt Kanban
  - TODO Week X Tasks #weekX
    - Sub: Code/Course
    - Sub: Content Creation (Post draft)
  - DOING Current
  - DONE Completed

- Journal Template
  - [[2026-01-20]] Progress: ... Learnings: ... Content Idea: ...
Monetization Note: ...

- Content Template
  - X Post Draft: Title: ... Text: ... Screenshot: ... Hashtags: ...
  - Promotion: Share link, tag communities.
  - Monetization: Idea for this week: ...

- Weekly Retro + Content Review
  - Wins: (include content engagement)
  - Improves:
  - Adjustments:
```

Это полный план с интеграцией — content не отвлекает, а усиливает мотивацию. Если нужно больше templates или уточнить монетизацию — скажи! 😊